`scripts/check-challenge-discrimination.sh` — User Guide

Last verified: 2026-08-26 (§6.J corpus floor — an unread Challenge corpus no longer prints ✓; LVA vacuous-pass sweep F7)
Inheritance: HelixConstitution §11.4.18 (script documentation mandate) + Lava §6.AB (Anti-Bluff Test-Suite Reinforcement, 27th §6.L invocation)

Overview

Mechanical enforcement of §6.AB.3 — every `Challenge*Test.kt` file MUST carry a falsifiability rehearsal block in its KDoc that names a deliberately-broken-but-non-crashing version of the production code AND the assertion message the test produces against that mutation. Without this, a test that passes against today's HEAD has no proof it would CATCH a future non-crashing regression — i.e. it could be a §6.J-spirit bluff.

Closes §6.AB-debt from deferred to wired (no Detekt setup required — bash scanner consistent with §6.AC pattern).

Two-layer enforcement (Layer 2 added 2026-05-15 from bluff-hunt audit)

Layer 1: KDoc marker check

Any one of these in the test file's KDoc satisfies Layer 1:

1. `FALSIFIABILITY REHEARSAL` (canonical, used by C00–C29; optionally prefixed with `§6.AB.3`)
2. `§6.AB-discrimination:` block (alternate canonical form)
3. Companion file at `.lava-ci-evidence/sp3a-challenges/<TestName>-<sha>.json` with a `falsifiability_rehearsal` / `discrimination` / `bluff_classification` field

Layer 2: BODY structural check

Files passing Layer 1 are also scanned for real-assertion patterns in the test body. A test that has the FALSIFIABILITY REHEARSAL marker but NO real assertion in its body is a §6.J spirit bluff: the doc claims discrimination, the body proves nothing.

Acceptable real-assertion patterns (any one is sufficient):

- Compose UI: `onNode|onAllNodes|assertIs|assertText|assertExists|fetchSemanticsNodes|composeRule\.waitUntil`
- JUnit assertions: `assertEquals|assertTrue|assertFalse|assertNotNull|assertSame|assertContains|assertThat`
- Classpath verification: `Class\.forName(...)` OR `::class\.java`
- Symbol references: `::<identifier>` (function-ref or class-ref proves the symbol exists at runtime)
- `check()` / `require()` calls

Falsifiability rehearsal (Layer 2):

- Mutation: synthetic `ChallengeBLUFF_REHEARSAL_DELETEMETest.kt` with the FALSIFIABILITY REHEARSAL block but no body assertions
- Observed: scanner reports "Layer 2 violations (marker present but body has no real assertion)" + names the file + lists remediation patterns
- Reverted: yes (synthetic file deleted; scanner returns to ✓ green)

Forensic anchor: `.lava-ci-evidence/bluff-hunt/2026-05-15-challenge-body-structural-audit.json` documented the manual audit that motivated this Layer 2 mechanical enforcement.

Usage

```
bash scripts/check-challenge-discrimination.sh
```

Default mode is STRICT (exit 1 on any violation). Set `LAVA_CHALLENGE_DISCRIMINATION_STRICT=0` to run in advisory mode.

Inputs

None (walks `app/src/androidTest/kotlin/lava/app/challenges/Challenge*Test.kt`).

Outputs

- One stdout summary block + per-violation list

- Exit 0 if 0 violations OR advisory mode; exit 1 in strict mode with violations

Side-effects

None — read-only scan.

Falsifiability rehearsal of the scanner itself

Test: scanner fires when a Challenge test's marker is stripped

- Mutation: `sed 's/FALSIFIABILITY REHEARSAL/[STRIPPED]/g'` on `Challenge00CrashSurvivalTest.kt`
- Observed: scanner reports "failing on 1 violation(s)" + lists the file path
- Reverted: yes (file restored; scanner reports 0 violations again)

Edge cases

Companion file with empty discrimination field

The scanner only checks for the field NAME, not its content. A future enhancement could parse the JSON and verify the field is non-empty + names a real mutation. For now: shipping `{"falsifiability_rehearsal": ""}` would silently pass — counted as a known limitation, tracked as §6.AB rolling improvement.

New Challenge test added without marker

Pre-push hook Layer 2 (`scripts/ci.sh --changed-only`) invokes this scanner. A commit that adds `app/src/androidTest/.../ChallengeXX_NoMarker.kt` will be REJECTED by pre-push under strict mode.

2026-08-26 update — §6.J corpus floor: the two ✓ lines can no longer be printed over an empty corpus

The defect

This scanner's success output is two universally quantified claims:

```
    ✓ all Challenge tests carry §6.AB.3 falsifiability rehearsal
documentation
    ✓ all Challenge test bodies contain real assertions (UI /
JUnit / classpath)
```

A universally quantified claim over an empty set is vacuously true. Both of the empty-corpus routes into this gate reported success:

Route	Old output	Old exit
`app/src/androidTest/kotlin/lava/app/challenges` absent	`==> §6.AB scan: no Challenge tests found (looked in ...)`	0
Directory present, zero `Challenge*Test.kt`	`Challenge tests: 0` **plus both ✓ lines**	0

The second is the sharper of the two: it prints two explicit claims *about all Challenge tests* having examined none of them. "Nothing was learned" reported as "nothing failed" is the shape §6.J forbids — and the same shape `scripts/check-constitution.sh`'s clause-6.H credential floor and `scripts/verify-all-constitution-rules.sh`'s registry floor already guard against elsewhere in this tree. This scanner simply had no equivalent.

The ordinary way to arrive at the first route is a `git clone` that never checked out `app/`, or a partial checkout; the second is reachable through a deletion or a bad merge that empties the directory without removing it.

What the scanner now refuses

Three refusals, all **exit 1**:

1. **Corpus directory absent** — `§6.AB VIOLATION: the Challenge-discrimination scan corpus directory is ABSENT.` Replaces the old exit-0 "no Challenge tests found" message entirely.
2. **Zero Challenge files examined** — `§6.AB VIOLATION: ... examined ZERO Challenge tests.` This check runs **before** the ✓ lines, precisely because those lines are the thing that must not be reachable over an empty corpus.
3. **Partial corpus** — `§6.AB VIOLATION: ... examined a PARTIAL corpus.` A floor that only fires at exactly zero is a floor with one stair: 73 declared and 2 present would otherwise pass just as cleanly as 73 and 73. The missing files are named individually, not just counted.

Every message states `Examined: <n>` against `Expected: <n>` so the corpus size is an explicit, reported fact rather than something the reader has to infer.

Why the expectation is derived from the git index

`Expected` comes from `git ls-files -- <challenge_dir>` filtered to `Challenge*Test.kt`, never from a literal number. The git index is the repository's own declaration of which Challenge files are supposed to exist, so it moves in lockstep with the corpus. A hardcoded count would go stale the moment a Challenge is added or removed — the same defect wearing a different mask.

Each refusal distinguishes its cause from the derived count: if the index declares Challenge files and the working tree has none, that is **working-tree drift** and the remedy is `git checkout -- <challenge_dir>`; if the index declares zero too, this is **not a Lava checkout** (or the scan is running from the wrong root) and the remedy is to re-run from the repository root. Two different situations, two different remedies, never one generic failure.

Two implementation notes worth keeping in mind if this block is edited:

- The count uses `awk`, not `grep -c`. `grep -c` exits 1 on a zero count, which under `set -e` inside a pipeline is its own hazard — the exact failure mode this sweep records.
- `git ls-files` is wrapped in `{ ...; } || true`. Outside a repository it exits 128, and under `set -euo pipefail` that would abort the script with **no message at all** — fail-closed, but with a diagnosis so empty it sends the reader nowhere. Degrading to a declared count of 0 lets the not-a-checkout branch say what actually happened.

Interaction with advisory mode

`LAVA_CHALLENGE_DISCRIMINATION_STRICT=0` still downgrades *violation* reporting to advisory. It does **not** downgrade the corpus floors: a scan that read nothing is not a scan whose findings can be advisory, because it has no findings either way.

§6.J falsifiability rehearsal of the floor itself

1. `mv app/src/androidTest/kotlin/lava/app/challenges /tmp/chal-hold` → run the scanner → expect **exit 1**, `corpus directory is ABSENT`, and `Expected: <n>` naming the count the git index declares. Before this change: exit 0.
2. `mkdir` the directory back empty → run → expect **exit 1**, `examined ZERO Challenge tests`, and **neither ✓ line printed**.
3. Restore a subset of the files → run → expect **exit 1**, `examined a PARTIAL corpus`, with the still-missing files named.
4. `mv /tmp/chal-hold ...` back in full → run → expect exit 0 and both ✓ lines.

Cross-references

- `scripts/check-non-fatal-coverage.sh` (sister scanner for §6.AC)
- `docs/helix-constitution-gates.md` (gate inventory)
- Lava `CLAUDE.md` §6.AB (the mandate)
- Lava `CLAUDE.md` §6.J (the parent anti-bluff principle)